

Informe de Arquitectura de microservicios: Greenbite

Integrantes: Fernanda Paredes - Martina Flores - Alexander Torres

Nombre del Docente: Rodrigo Alejandro Chacon Moreno

Nombre Asignatura Sección:

Desarrollo fullstack III 001D

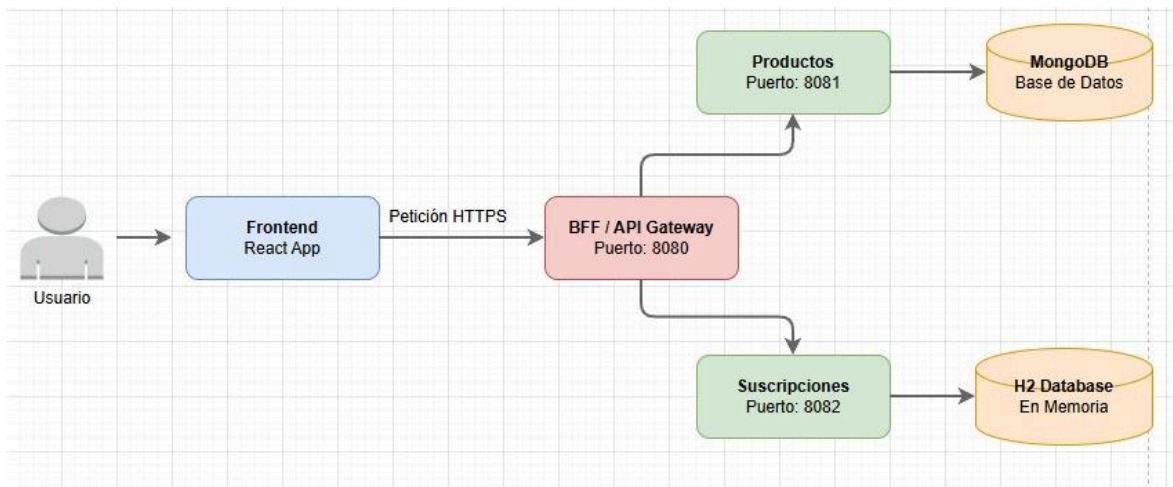
1. introducción y objetivo del sistema

El presente documento detalla la solución arquitectónica implementada para la plataforma GreenBite. El objetivo principal de esta arquitectura es resolver los problemas de acoplamiento, escalabilidad y disponibilidad mediante la división de las responsabilidades del negocio en componentes autónomos, distribuidos y altamente cohesivos. La propuesta integra un frontend moderno, una capa de abstracción para la optimización de las peticiones de cliente y dos servicios backend independientes con almacenamiento de datos especializado.

2. Esquema de la Arquitectura de Microservicios

El sistema **GreenBite** está diseñado bajo un enfoque de microservicios desacoplados, estructurado en tres capas principales:

1. **Capa de Cliente (Frontend):** Una SPA (Single Page Application) desarrollada en **React + Vite** que corre en el puerto 5173. No conoce las direcciones IP ni los puertos internos de los microservicios backend.
2. **Capa de Mediación (BFF / API Gateway):** Un componente centralizado construido con **Spring Cloud Gateway** en el puerto 8080. Es el único punto de contacto para el Frontend.
3. **Capa de Servicios de Negocio (Backend):** Dos microservicios autónomos que resuelven dominios específicos:
 - a. **Microservicio Productos:** Puerto 8081, persistencia en **MongoDB**.
 - b. **Microservicio Suscripciones:** Puerto 8082, persistencia en **H2 Database**.



3. Especificaciones de componentes

La siguiente matriz detalla la distribución de puertos, tecnologías principales y el propósito arquitectónico de cada componente desarrollado para la solución:

Componente	Tipo	Puerto	stack Tecnológico	Propósito Arquitectónico
GreenBite - Frontend	Cliente Web (SPA)	5173	React, Vite, JavaScript, Axios, CSS3	Proporcionar una interfaz gráfica intuitiva para el usuario final que permita consumir las operaciones CRUD de productos y planes.
BFF / API Gateway	Pasarela de API	8080	Java 17, Spring Boot 3.4.1, Spring Cloud Gateway, Spring WebFlux, Maven	Centralizar las peticiones del cliente, abstraer puertos internos del backend y mitigar problemas de acoplamiento.
Microservicio Productos	Servicio de Dominio	8081	Java 17, Spring Boot 3.4.1, Spring Data MongoDB, Swagger, JUnit 5, Mockito, JaCoCo	Gestionar el ciclo de vida completo (creación, lectura y eliminación) del catálogo de productos disponibles.
Microservicio Suscripciones	Servicio de Dominio	8082	Java 17, Spring Boot 3.4.1, Spring Data JPA, H2 Database, Swagger, JUnit 5, Mockito, JaCoCo	Gestionar los planes y suscripciones de usuarios mediante transacciones estructuradas y aisladas.

4. Justificación de los patrones de diseño implementados:

La propuesta de solución aplica de forma creativa patrones arquitectónicos modernos orientados a optimizar el rendimiento y la mantenibilidad del software:

1. Patrón Backend For Frontend (BFF)

Implementado en la capa de mediación (puerto 8080) como interfaz directa para la aplicación React. Al centralizar las llamadas en un BFF, se evita que el cliente web deba realizar conexiones múltiples a distintos servidores o puertos. Esto reduce la latencia en redes móviles y aísla al cliente de los cambios internos de la arquitectura backend: si un microservicio cambia su puerto de despliegue, solo se actualiza la regla en el BFF, sin necesidad de recompilar el Frontend.

2. Patrón API Gateway

Configurado mediante **Spring Cloud Gateway** utilizando routing dinámico y predicados basados en rutas HTTP. Proporciona un punto único de entrada para aplicar políticas transversales uniformes en el futuro (tales como autenticación JWT, rate limiting o logs centralizados), protegiendo la infraestructura interna del sistema de exposiciones directas a internet.

3. Patrón Database per Service

El Microservicio de Productos maneja exclusivamente persistencia en **MongoDB**, mientras que el Microservicio de Suscripciones gestiona sus datos a través de **H2 Database**. Rompe con el antipatrón de la base de datos compartida (monolito de datos). Al dar a cada microservicio el control total de su persistencia, se elimina el riesgo de que una caída en el esquema de productos afecte al módulo de suscripciones, garantizando un acoplamiento laxo y una alta tolerancia a fallos en producción.

5. Estructura de la solución y control de versiones:

De acuerdo con las buenas prácticas de desarrollo ágil y los estándares exigidos para el encargo, la solución se distribuye en repositorios independientes dentro de **GitHub** para salvaguardar la modularidad del código fuente:

GreenBite-Ecosistema:

- **Repositorio-Principal/** -> Documentación de arquitectura, diagramas e informes (PDFs)
- **Repositorio-Frontend/** -> Proyecto React + Vite (Estándar NPM, package.json, src/)
- **Repositorio-MS-Productos/** -> Microservicio Java/Spring Boot + configuración MongoDB (pom.xml)
- **Repositorio-MS-Suscripciones/** -> Microservicio Java/Spring Boot + configuración H2 JPA (pom.xml).

Cada uno de los repositorios de código cuenta con su correspondiente archivo README.md que detalla los prerequisites (Java 17, Node, Maven), variables de entorno, scripts de inicialización (npm run dev, mvn spring-boot:run) y guías de despliegue local para asegurar la replicabilidad del entorno.